

PySpark auf Databricks

PySpark ist die Python-Schnittstelle zu Apache Spark und in der Praxis die am häufigsten genutzte Art, mit Spark zu arbeiten. Diese Seite erklärt die zentralen Grundbegriffe – DataFrames, Transformationen, Actions und Lazy Evaluation – und zeigt, welche weiteren APIs über PySpark zugänglich sind.

Databricks und Apache Spark

Databricks baut direkt auf Apache Spark auf, einer vereinheitlichten Analyse-Engine für Big Data und maschinelles Lernen. PySpark macht diese Engine über eine vertraute Python-Syntax nutzbar.

DataFrames – die zentrale Datenstruktur

Ein DataFrame ist das wichtigste Objekt in Spark: eine zweidimensionale, beschriftete Datenstruktur, ähnlich einer Tabelle in einer Tabellenkalkulation. Ein DataFrame besteht aus drei Bestandteilen:

- **Schema:** Legt fest, welche Spalten es gibt und welchen Datentyp jede Spalte hat.
- **Reihen (Rows):** Jede Reihe entspricht einem einzelnen Datensatz.
- **Spalten (Columns):** Können sowohl einfache Datentypen (Zahlen, Text) als auch komplexe, verschachtelte Datentypen enthalten.

Wie Datenverarbeitung in PySpark abläuft

Transformationen

Transformationen sind Anweisungen, um Daten zu laden oder zu verändern – etwa `spark.read` zum Einlesen von Daten, oder Operationen wie Joins und Aggregationen. Wichtig: Eine Transformation verändert nicht direkt Daten, sondern beschreibt nur, was passieren soll.

Lazy Evaluation (verzögerte Ausführung)

Ein zentrales Prinzip von Spark: "Spark does not act on transformations until actions are called" – Spark führt Transformationen erst dann tatsächlich aus, wenn eine Aktion aufgerufen wird.

Dadurch kann Spark die gesamte Kette an Transformationen im Vorfeld analysieren und optimieren, bevor überhaupt Rechenleistung verbraucht wird.

Actions (Aktionen)

Actions lösen die tatsächliche Berechnung aus und liefern ein konkretes Ergebnis zurück, zum Beispiel:

- `display` – Ergebnis anzeigen
- `count` – Anzahl der Zeilen ermitteln
- `saveAsTable` – Ergebnis als Tabelle speichern

Immutabilität

DataFrames sind unveränderlich (immutable): Jede Transformation erzeugt einen komplett neuen DataFrame, statt den bestehenden zu verändern. Das macht Spark-Code vorhersehbarer und erleichtert die parallele Verarbeitung, weil sich niemand "unter der Hand" gemeinsam genutzte Daten verändert.

Weitere APIs, die über PySpark zugänglich sind

PySpark ist nicht nur DataFrames – darüber hinaus stehen weitere Bausteine des Spark-Ökosystems zur Verfügung:

- **Spark SQL und DataFrames:** SQL-artige Abfragen direkt auf DataFrames.
- **Structured Streaming:** Verarbeitung kontinuierlicher Datenströme (siehe eigenes Kapitel).
- **Pandas API on Spark:** Aus pandas bekannte Syntax auf verteilten Datenmengen (siehe eigenes Kapitel).
- **MLlib:** Verteiltes maschinelles Lernen (siehe eigenes Kapitel).
- **GraphX:** Verarbeitung und Analyse von Graphdaten (z. B. Netzwerke, Beziehungen zwischen Entitäten).

Lernressourcen

Für den praktischen Einstieg verlinkt die Seite auf ein DataFrames-Tutorial sowie auf einen eigenen Leitfaden zu "PySpark-Grundlagen". Zusätzlich wird auf die offizielle Apache-Spark-Dokumentation verwiesen.

Warum PySpark der naheliegende Startpunkt ist

Für die meisten Kursteilnehmer ist PySpark der praktischste Einstieg in Spark, weil Python ohnehin weit verbreitet ist und die PySpark-API sich eng an vertraute Konzepte aus Datenanalyse-Bibliotheken wie pandas anlehnt. Wer die Grundprinzipien Lazy Evaluation, Transformationen/Actions und Immutabilität einmal verstanden hat, findet sich auch in den anderen Sprach-APIs (Scala, R) schnell zurecht.

Quelle: <https://docs.databricks.com/aws/en/pyspark/>

Stand: 2026-08-06